

React moderno na prática

Módulo: 09

Aula: 11 - Revisão do módulo + Provider com tema Light/Dark

Instrutor: Lucca Dumas

Duração: 2 h

CONTEÚDO

1) Abertura da aula

Esta aula fecha o Módulo 09 com revisão dos conteúdos já estudados e uma aplicação final no projeto.

A aplicação final é feita com **Context + Provider**, usando alternância de tema **Light/Dark** na página `administracao/usuarios`.

2) O que veremos hoje?

1. Revisão dos conteúdos das aulas anteriores
 2. Mapa consolidado do fluxo de desenvolvimento no projeto
 3. Aplicação final com `TemaContexto`, `TemaProvider` e botão na tela de usuários
 4. Prática guiada de fechamento do módulo
-

3) Revisão da Aula 05

- Hooks essenciais: `useState`, `useEffect`, `useContext`;
- diferença entre `DOM` e `ReactDOM`;
- `export function` e `export default`;
- introdução de `Context + Provider`;
- comandos com `Yarn` para execução do projeto.

Exemplo:

```
const [contador, setContador] = useState(0);

useEffect(() => {
  document.title = `Valor: ${contador}`;
}, [contador]);
```

4) Revisão da Aula 06

- `Context + Provider` para compartilhamento de estado;
- comparação entre `Context` e `Zustand`;
- leitura de estado global no projeto;
- providers na árvore principal do app.

Exemplo:

```
const ToastContext = createContext(undefined);

export function ToastProvider({ children }) {
  const [toasts, setToasts] = useState([]);
  return <ToastContext.Provider value={{ toasts, setToasts }}>{children}</ToastContext.Provider>;
}
```

5) Revisão da Aula 07

- `Context/Provider` compartilhando dados entre páginas;
- hook customizado para consumo de contexto;

- estrutura de arquivos para contexto, provider e hook.

Exemplo:

```
export function useMensagem() {
  const contexto = useContext(MensagemContexto);
  if (!contexto) {
    throw new Error('useMensagem deve ser usado dentro de MensagemProvider');
  }
  return contexto;
}
```

6) Revisão da Aula 08

- Navegação com `react-router-dom`;
- definição de rotas e links;
- consumo de API com `fetch` e `axios`;
- estado de carregamento e tratamento de erro.

Exemplo:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/usuarios" element={<Usuarios />} />
</Routes>
```

7) Revisão da Aula 09

- Leitura de tela real no projeto (`screen` -> `service` -> tabela);
- integração de dados na listagem de usuários;
- navegação entre listagem e cadastro.

Exemplo:

```
useEffect(() => {
  async function carregarUsuarios() {
    const resposta = await usuariosEndpoints.pesquisarUsuarios({ pagina: 1, tamanho: 20 });
    setListaUsuarios(resposta.dados ?? []);
  }
  carregarUsuarios();
}, []);
```

8) Revisão da Aula 10

- Introdução a Jest e React Testing Library;
- leitura de testes existentes no projeto;
- reforço de práticas integradas no código real.

Exemplo:

```
it('deve chamar a função ao clicar', () => {
  const aoClicar = jest.fn();
  render(<Button onClick={aoClicar}>Enviar</Button>);
  fireEvent.click(screen.getByText('Enviar'));
  expect(aoClicar).toHaveBeenCalledTimes(1);
});
```

9) Mapa consolidado do módulo

Fluxo consolidado:

1. entender estrutura e padrão do projeto;
2. trabalhar estado local e efeitos;
3. compartilhar estado quando necessário com Context/Provider;
4. navegar entre telas e consumir API;
5. renderizar dados na interface com consistência;
6. validar qualidade local com lint e práticas de revisão.

10) Aplicação final: Provider com tema Light/Dark

Contexto e Provider

Arquivo: `src/contexts/TemaContexto.tsx`

```
import { createContext, useContext, useState, type ReactNode } from 'react';

type Tema = 'light' | 'dark';

type TemaContextoTipo = {
  tema: Tema;
  alternarTema: (novoTema: Tema) => void;
};

const TemaContexto = createContext<TemaContextoTipo>(undefined);

export function TemaProvider({ children }: { children: ReactNode }) {
  const [tema, setTema] = useState<Tema>('light');

  const alternarTema = (novoTema: Tema) => {
    setTema(novoTema);
  };

  return <TemaContexto.Provider value={{ tema, alternarTema }}>{children}</TemaContexto.Provider>;
}

export function useTema() {
  const contexto = useContext(TemaContexto);
  if (!contexto) {
    throw new Error('useTema deve ser usado dentro de TemaProvider');
  }
  return contexto;
}
```

Provider na aplicação

Arquivo: `src/App.tsx`

```
function App() {
  return (
    <TemaProvider>
      <ToastProvider>
        <CustomToastProvider>
          <AppContent />
        </CustomToastProvider>
      </ToastProvider>
    </TemaProvider>
  );
}
```

Botão na tela de usuários

Arquivo: `src/screens/Administracao/Usuarios/index.tsx`

```
const { tema, alternarTema } = useTema();

<Button
  variant="outline"
  size="md"
  onClick={() => alternarTema(tema === 'dark' ? 'light' : 'dark')}
  data-testid="usuarios-botao-alternar-tema"
>
  {tema === 'dark' ? 'Desativar modo escuro' : 'Ativar modo escuro'}
</Button>
```

11) Hands-on da aula

1. Revisar os pontos das aulas 05 a 10.
2. Abrir `src/contexts/TemaContexto.tsx` e validar estrutura do contexto.
3. Abrir `src/App.tsx` e validar o `TemaProvider` envolvendo o app.
4. Abrir `src/screens/Administracao/Usuarios/index.tsx` e validar o botão de tema.
5. Executar:

```
yarn lint
yarn dev
```

12) Exercícios de fixação

1. Qual é a função do `Provider` no Context? () Criar rotas (X) Disponibilizar estado compartilhado para os componentes filhos () Substituir o `useEffect` () Fazer request HTTP
2. Em qual arquivo o `TemaProvider` envolve o conteúdo principal? () `src/main.css` (X) `src/App.tsx` () `src/services/endpoints/usuarios.js` () `src/components/buttons/button.tsx`
3. O que o botão na tela de usuários faz? () Recarrega a tabela () Navega para cadastro (X) Alterna o valor de tema entre `light` e `dark` () Fecha o modal

13) Desafio para casa

1. Descrever o fluxo:
 - `TemaContexto` -> `TemaProvider` -> `useTema` -> botão da tela.
2. Confirmar na tela de usuários:
 - o clique altera o texto do botão entre ativar/desativar.
3. Rodar:

```
yarn lint
```

Regras

- manter padrão de organização dos arquivos do projeto;
- não alterar contratos de endpoints;
- manter escopo no contexto de tema e tela de usuários.